Task4 - Binary Classification

Runxin Shen

2026-11-10

Step 1 load and split the data

```
library(tidyverse)
library(tidymodels)
library(here)

# Load labeled training data
load(here("data", "claims-raw.RData"))

# Load unlabeled final test data
load(here("data", "claims-test.RData"))

# Check the number of observations
stopifnot(nrow(claims_raw) == 2165)
stopifnot(nrow(claims_test) == 929)

# Check for duplicated IDs
stopifnot(!anyDuplicated(claims_raw$.id))
stopifnot(!anyDuplicated(claims_test$.id))

# Check for missing HTML
stopifnot(!anyNA(claims_raw$text_tmp))
stopifnot(!anyNA(claims_test$text_tmp))

# Inspect binary class counts
class_counts <- table(claims_raw$bclass)
print(class_counts)
```

```
##
## N/A: No relevant content.    Relevant claim content
##                      1032                      1133
```

```
# Inspect binary class proportions
class_proportions <- prop.table(class_counts)
print(class_proportions)
```

```
##
## N/A: No relevant content.    Relevant claim content
##                 0.4766744                 0.5233256
```

```r
# Create a stratified training and validation split
set.seed(102926)

data_split <- initial_split(
  claims_raw,
  prop = 0.8,
  strata = bclass
)

train_raw <- training(data_split)
valid_raw <- testing(data_split)

# Check the number of pages in each set
split_summary <- tibble(
  dataset = c("Training", "Validation"),
  n_pages = c(nrow(train_raw), nrow(valid_raw))
)

print(split_summary)
```

```
## # A tibble: 2 x 2
##   dataset    n_pages
##   <chr>        <int>
## 1 Training      1731
## 2 Validation     434
```

```r
# Check class proportions in the training set
train_class_proportions <- prop.table(
  table(train_raw$bclass)
)

print(train_class_proportions)
```

```
##
## N/A: No relevant content.    Relevant claim content
##                 0.4766031                 0.5233969
```

```r
# Check class proportions in the validation set
valid_class_proportions <- prop.table(
  table(valid_raw$bclass)
)

print(valid_class_proportions)
```

```
##
## N/A: No relevant content.    Relevant claim content
##                 0.4769585                 0.5230415
```

```r
# Confirm that the training and validation sets do not overlap
shared_ids <- intersect(
  train_raw$.id,
```

```
  valid_raw$.id
)

stopifnot(length(shared_ids) == 0)

# Confirm that all labeled pages are retained
all_split_ids <- unique(
  c(train_raw$.id, valid_raw$.id)
)

stopifnot(
  length(all_split_ids) == nrow(claims_raw)
)
```

Step 2 extract and clean webpage text

```
library(rvest)

# Extract text from HTML
extract_page_text <- function(html) {
  tryCatch(
    {
      document <- read_html(html)

      page_text <- document |>
        html_elements(
          "title, h1, h2, h3, h4, h5, h6, p"
        ) |>
        html_text2() |>
        str_c(collapse = " ") |>
        str_squish()

      if (str_length(page_text) == 0) {
        page_text <- document |>
          html_text2() |>
          str_squish()
      }

      page_text
    },
    error = function(e) {
      ""
    }
  )
}

clean_page_text <- function(text) {
  text |>
    str_replace_all(
      "https?://\\S+|www\\.\\S+",
      " "
```

```
    ) |>
    str_replace_all(
      "\\b[[:alnum:]._%+-]+@[[:alnum:].-]+\\.[[:alpha:]]{2,}\\b",
      " "
    ) |>
    str_replace_all(
      "([a-z])([A-Z])",
      "\\1 \\2"
    ) |>
    str_to_lower() |>
    str_replace_all(
      "[^a-z0-9\\s]",
      " "
    ) |>
    str_squish()
}

preprocess_pages <- function(data) {
  data |>
    mutate(
      text_clean = map_chr(
        text_tmp,
        function(html) {
          extracted_text <- extract_page_text(html)
          clean_page_text(extracted_text)
        }
      )
    )
}

# Apply text preprocessing to each dataset

train_text <- preprocess_pages(train_raw)

valid_text <- preprocess_pages(valid_raw)

test_text <- preprocess_pages(claims_test)
```

```
# Check the number of pages

nrow(train_text)
```

```
## [1] 1731
```

```
nrow(valid_text)
```

```
## [1] 434
```

```
nrow(test_text)
```

```
## [1] 929
```

```r
# Confirm that no pages were removed

stopifnot(nrow(train_text) == nrow(train_raw))

stopifnot(nrow(valid_text) == nrow(valid_raw))

stopifnot(nrow(test_text) == nrow(claims_test))
```

```r
# Preview part of one cleaned webpage

str_sub(
  train_text$text_clean[1],
  start = 1,
  end = 500
)
```

```
## [1] "new mugshots january 3 2014 miami dade county fl mugshots find the truth filter by location fil
```

Step 3A Tokenization

```r
library(tidytext)

tokenize_pages <- function(data) {
  data |>
    select(.id, text_clean) |>
    unnest_tokens(
      output = term,
      input = text_clean,
      token = "words"
    ) |>
    filter(
      str_detect(term, "^[a-z]+$"),
      str_length(term) > 2) |>
    anti_join(
      stop_words,
      by = c("term" = "word")
    ) |>
    count(
      .id,
      term,
      name = "n"
    )
}

# Tokenize each dataset
train_counts <- tokenize_pages(train_text)
valid_counts <- tokenize_pages(valid_text)
test_counts <- tokenize_pages(test_text)

# Preview training word counts
```

```
train_counts |>
  slice_head(n = 10)
```

```
## # A tibble: 10 x 3
##    .id   term          n
##    <chr> <chr>     <int>
##  1 url1  access        2
##  2 url1  addresses     1
##  3 url1  agree         1
##  4 url1  app           1
##  5 url1  blocks        1
##  6 url1  business      2
##  7 url1  canadian      1
##  8 url1  charge        1
##  9 url1  community     3
## 10 url1  company       1
```

```
# Check the number of unique training words
n_distinct(train_counts$term)
```

```
## [1] 36910
```

Step 3B create the training vocabulary

```
n_train_pages <- nrow(train_text)

vocabulary <- train_counts |>
  count(
    term,
    name = "document_frequency"
  ) |>
  filter(
    document_frequency >= 5
  ) |>
  mutate(
    idf = log(
      n_train_pages / document_frequency
    )
  ) |>
  arrange(term)

# Check vocabulary size

nrow(vocabulary)
```

```
## [1] 7345
```

```r
# Preview the vocabulary

vocabulary |>
  slice_head(n = 10)
```

```
## # A tibble: 10 x 3
##    term      document_frequency   idf
##    <chr>                  <int> <dbl>
##  1 aaa                       13  4.89
##  2 aaron                     26  4.20
##  3 abandoned                  7  5.51
##  4 abbott                     7  5.51
##  5 abby                       7  5.51
##  6 abc                        8  5.38
##  7 abigail                    5  5.85
##  8 ability                   35  3.90
##  9 aboard                     5  5.85
## 10 abruptly                   5  5.85
```

Step 3C create TF IDF matrices

```r
library(Matrix)

build_tfidf_matrix <- function(
  counts,
  page_ids,
  vocabulary
) {
  weighted_counts <- counts |>
    group_by(.id) |>
    mutate(
      tf = n / sum(n)
    ) |>
    ungroup() |>
    inner_join(
      vocabulary |>
        select(term, idf),
      by = "term"
    ) |>
    mutate(
      tf_idf = tf * idf
    )

  sparseMatrix(
    i = match(
      weighted_counts$.id,
      page_ids
    ),
    j = match(
      weighted_counts$term,
```

```
      vocabulary$term
    ),
    x = weighted_counts$tf_idf,
    dims = c(
      length(page_ids),
      nrow(vocabulary)
    ),
    dimnames = list(
      page_ids,
      vocabulary$term
    )
  )
}

# Create the training matrix

x_train <- build_tfidf_matrix(
  counts = train_counts,
  page_ids = train_text$.id,
  vocabulary = vocabulary
)

# Create the validation matrix

x_valid <- build_tfidf_matrix(
  counts = valid_counts,
  page_ids = valid_text$.id,
  vocabulary = vocabulary
)

# Check matrix dimensions

dim(x_train)
```

```
## [1] 1731 7345
```

```
dim(x_valid)
```

```
## [1]  434 7345
```

```
# Check row alignment

stopifnot(
  identical(
    rownames(x_train),
    train_text$.id
  )
)

stopifnot(
  identical(
    rownames(x_valid),
```

```
    valid_text$.id
  )
)

# Check column alignment

stopifnot(
  identical(
    colnames(x_train),
    colnames(x_valid)
  )
)

# Count pages with no retained terms

sum(Matrix::rowSums(x_train) == 0)
```

```
## [1] 13
```

```
sum(Matrix::rowSums(x_valid) == 0)
```

```
## [1] 4
```

```
# Create outcome vectors

y_train <- factor(
  train_text$bclass
)

y_valid <- factor(
  valid_text$bclass,
  levels = levels(y_train)
)

# Check outcome levels

levels(y_train)
```

```
## [1] "N/A: No relevant content." "Relevant claim content"
```

```
# Check outcome lengths

length(y_train)
```

```
## [1] 1731
```

```
length(y_valid)
```

```
## [1] 434
```

```r
# Confirm alignment

stopifnot(length(y_train) == nrow(x_train))
stopifnot(length(y_valid) == nrow(x_valid))
```

Step 4 train and tune the binary model

```r
library(glmnet)

# Create stratified cross validation folds

set.seed(102926)

n_folds <- 5

fold_id <- integer(
  length(y_train)
)

for (class_label in levels(y_train)) {
  class_rows <- which(
    y_train == class_label
  )

  fold_id[class_rows] <- sample(
    rep(
      1:n_folds,
      length.out = length(class_rows)
    )
  )
}

# Check fold class counts

table(
  fold_id,
  y_train
)
```

```
##        y_train
## fold_id N/A: No relevant content. Relevant claim content
##       1                       165                    182
##       2                       165                    181
##       3                       165                    181
##       4                       165                    181
##       5                       165                    181
```

```r
# Define alpha values

alpha_values <- c(
```

```
  0,
  0.25,
  0.5,
  0.75,
  1
)

# Fit one cross validation model for each alpha

cv_models <- lapply(
  alpha_values,
  function(alpha_value) {
    cv.glmnet(
      x = x_train,
      y = y_train,
      family = "binomial",
      alpha = alpha_value,
      foldid = fold_id,
      type.measure = "class"
    )
  }
)

# Summarize cross validation results

cv_summary <- map2_dfr(
  alpha_values,
  cv_models,
  function(alpha_value, model) {
    best_position <- which.min(
      model$cvm
    )

    tibble(
      alpha = alpha_value,
      lambda = model$lambda[best_position],
      cv_error = model$cvm[best_position],
      cv_accuracy = 1 - model$cvm[best_position]
    )
  }
)

print(cv_summary)
```

```
## # A tibble: 5 x 4
##   alpha lambda cv_error cv_accuracy
##   <dbl>  <dbl>    <dbl>       <dbl>
## 1  0    7.26      0.215       0.785
## 2  0.25 0.0847    0.208       0.792
## 3  0.5  0.0444    0.205       0.795
## 4  0.75 0.0282    0.206       0.794
## 5  1    0.0168    0.206       0.794
```

```r
# Select the best alpha

best_model_index <- which.max(
  cv_summary$cv_accuracy
)

best_alpha <- cv_summary$alpha[
  best_model_index
]

best_lambda <- cv_summary$lambda[
  best_model_index
]

best_cv_model <- cv_models[[
  best_model_index
]]

# Print selected parameters

best_alpha
```

```
## [1] 0.5
```

```r
best_lambda
```

```
## [1] 0.04435743
```

Step 5 evaluate the validation data

```r
# Confirm class order

expected_levels <- c(
  "N/A: No relevant content.",
  "Relevant claim content"
)

stopifnot(
  identical(
    levels(y_train),
    expected_levels
  )
)

negative_class <- levels(y_train)[1]
positive_class <- levels(y_train)[2]

# Predict validation probabilities

valid_probabilities <- predict(
```

```
  best_cv_model,
  newx = x_valid,
  s = best_lambda,
  type = "response"
) |>
  as.numeric()

# Convert probabilities to class predictions

valid_predictions <- if_else(
  valid_probabilities >= 0.5,
  positive_class,
  negative_class
)

valid_predictions <- factor(
  valid_predictions,
  levels = levels(y_train)
)

# Store validation results

validation_results <- tibble(
  .id = valid_text$.id,
  truth = y_valid,
  estimate = valid_predictions,
  probability = valid_probabilities
)

# Check prediction output

stopifnot(
  nrow(validation_results) ==
    nrow(valid_text)
)

stopifnot(
  !anyNA(validation_results$estimate)
)

stopifnot(
  !anyNA(validation_results$probability)
)

# Calculate validation metrics

validation_metrics <- bind_rows(
  accuracy(
    validation_results,
    truth = truth,
    estimate = estimate
  ),
  sens(
```

```
    validation_results,
    truth = truth,
    estimate = estimate,
    event_level = "second"
  ),
  spec(
    validation_results,
    truth = truth,
    estimate = estimate,
    event_level = "second"
  ),
  roc_auc(
    validation_results,
    truth = truth,
    probability,
    event_level = "second"
  )
)

print(validation_metrics)
```

```
## # A tibble: 4 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.802
## 2 sens     binary         0.767
## 3 spec     binary         0.841
## 4 roc_auc  binary         0.863
```

```
# Create a confusion matrix

validation_confusion <- conf_mat(
  validation_results,
  truth = truth,
  estimate = estimate
)

validation_confusion
```

```
##                            Truth
## Prediction                  N/A: No relevant content. Relevant claim content
##   N/A: No relevant content.                      174                     53
##   Relevant claim content                          33                    174
```

Step 6A create the final training data

```
# Process all labeled pages

full_text <- preprocess_pages(
  claims_raw
```

```
)

# Tokenize all labeled pages

full_counts <- tokenize_pages(
  full_text
)

# Create the final vocabulary

n_full_pages <- nrow(full_text)

final_vocabulary <- full_counts |>
  count(
    term,
    name = "document_frequency"
  ) |>
  filter(
    document_frequency >= 5
  ) |>
  mutate(
    idf = log(
      n_full_pages / document_frequency
    )
  ) |>
  arrange(term)

# Check the final vocabulary size

nrow(final_vocabulary)
```

```
## [1] 8415
```

```
# Create the full training matrix

x_full <- build_tfidf_matrix(
  counts = full_counts,
  page_ids = full_text$.id,
  vocabulary = final_vocabulary
)

# Create the final test matrix

x_test <- build_tfidf_matrix(
  counts = test_counts,
  page_ids = test_text$.id,
  vocabulary = final_vocabulary
)

# Create the full outcome vector

y_full <- factor(
  full_text$bclass,
```

```
  levels = levels(y_train)
)

# Check matrix dimensions

dim(x_full)
```

```
## [1] 2165 8415
```

```
dim(x_test)
```

```
## [1]  929 8415
```

```
# Check outcome length

length(y_full)
```

```
## [1] 2165
```

```
# Check row alignment

stopifnot(
  identical(
    rownames(x_full),
    full_text$.id
  )
)

stopifnot(
  identical(
    rownames(x_test),
    test_text$.id
  )
)

# Check column alignment

stopifnot(
  identical(
    colnames(x_full),
    colnames(x_test)
  )
)

# Check outcome alignment

stopifnot(
  length(y_full) == nrow(x_full)
)

# Check the number of test pages
```

```r
stopifnot(
  nrow(x_test) == 929
)

# Count pages with no retained terms

sum(Matrix::rowSums(x_full) == 0)
```

```
## [1] 17
```

```r
sum(Matrix::rowSums(x_test) == 0)
```

```
## [1] 7
```

Step 6B fit the final binary model

```r
# Check the selected alpha

best_alpha
```

```
## [1] 0.5
```

```r
stopifnot(
  length(best_alpha) == 1,
  !is.na(best_alpha)
)

# Create stratified cross validation folds

set.seed(102926)

final_n_folds <- 5

final_fold_id <- integer(
  length(y_full)
)

for (class_label in levels(y_full)) {
  class_rows <- which(
    y_full == class_label
  )

  final_fold_id[class_rows] <- sample(
    rep(
      1:final_n_folds,
      length.out = length(class_rows)
    )
  )
```

```
}

# Check class counts in each fold

table(
  final_fold_id,
  y_full
)
```

```
##              y_full
## final_fold_id N/A: No relevant content. Relevant claim content
##             1                       207                     227
##             2                       207                     227
##             3                       206                     227
##             4                       206                     226
##             5                       206                     226
```

```
# Select the final lambda

final_cv_model <- cv.glmnet(
  x = x_full,
  y = y_full,
  family = "binomial",
  alpha = best_alpha,
  foldid = final_fold_id,
  type.measure = "class"
)

final_lambda <- final_cv_model$lambda.min

# Print final parameters

best_alpha
```

```
## [1] 0.5
```

```
final_lambda
```

```
## [1] 0.02711308
```

```
# Fit the final model using all labeled pages

final_binary_model <- glmnet(
  x = x_full,
  y = y_full,
  family = "binomial",
  alpha = best_alpha,
  lambda = final_lambda
)

# Check the final model

final_binary_model
```

```
##
## Call:  glmnet(x = x_full, y = y_full, family = "binomial", alpha = best_alpha,      lambda = final_la
##
##    Df  %Dev  Lambda
## 1 509 52.71 0.02711
```

```
# Count nonzero coefficients

final_coefficients <- coef(
  final_binary_model,
  s = final_lambda
)

sum(final_coefficients != 0)
```

```
## [1] 510
```

Step 7 generate binary predictions

```
# Predict probabilities for the final test data

test_probabilities <- predict(
  final_binary_model,
  newx = x_test,
  type = "response"
) |>
  as.numeric()

# Check predicted probabilities

stopifnot(
  length(test_probabilities) == 929
)

stopifnot(
  !anyNA(test_probabilities)
)

stopifnot(
  all(
    test_probabilities >= 0 &
      test_probabilities <= 1
  )
)

# Define class labels

negative_class <- levels(y_full)[1]
positive_class <- levels(y_full)[2]

negative_class
```

```
## [1] "N/A: No relevant content."
```

```
positive_class
```

```
## [1] "Relevant claim content"
```

```
# Convert probabilities to class predictions

test_predictions <- if_else(
  test_probabilities >= 0.5,
  positive_class,
  negative_class
)

test_predictions <- factor(
  test_predictions,
  levels = levels(y_full)
)

# Create the binary prediction data frame

binary_pred_df <- tibble(
  .id = test_text$.id,
  bclass.pred = test_predictions
)

# Check the binary predictions

stopifnot(
  nrow(binary_pred_df) == 929
)

stopifnot(
  !anyDuplicated(binary_pred_df$.id)
)

stopifnot(
  !anyNA(binary_pred_df$bclass.pred)
)

stopifnot(
  identical(
    binary_pred_df$.id,
    claims_test$.id
  )
)

# Preview predictions

binary_pred_df |>
  slice_head(n = 10)
```

```
## # A tibble: 10 x 2
```

```
##    .id   bclass.pred
##    <chr> <fct>
##  1 url3  Relevant claim content
##  2 url6  Relevant claim content
##  3 url8  N/A: No relevant content.
##  4 url9  N/A: No relevant content.
##  5 url11 N/A: No relevant content.
##  6 url16 N/A: No relevant content.
##  7 url20 N/A: No relevant content.
##  8 url23 N/A: No relevant content.
##  9 url25 N/A: No relevant content.
## 10 url27 Relevant claim content
```

```r
# Check predicted class counts

table(
  binary_pred_df$bclass.pred
)
```

```
##
## N/A: No relevant content.    Relevant claim content
##                       498                       431
```

Step 8A save the binary model and predictions

```r
# Create a deployable model bundle

binary_model_bundle <- list(
  model_type = "Elastic net logistic regression",
  fit = final_binary_model,
  vocabulary = final_vocabulary,
  alpha = best_alpha,
  lambda = final_lambda,
  threshold = 0.5,
  class_levels = levels(y_full),
  minimum_document_frequency = 5,
  preprocessing = list(
    html_elements = "title, h1, h2, h3, h4, h5, h6, p",
    minimum_term_length = 3,
    alphabetic_tokens_only = TRUE,
    remove_stop_words = TRUE,
    feature_type = "Unigram TF IDF"
  ),
  validation_metrics = validation_metrics
)

# Save the model bundle

saveRDS(
  binary_model_bundle,
  here(
```

```
    "results",
    "task4-binary-model.rds"
  )
)

# Save the binary predictions

save(
  binary_pred_df,
  file = here(
    "results",
    "task4-binary-predictions.RData"
  )
)
```